Factory Pattern (Factory Method and Abstract Factory)

Theory, Edge Cases & Code Examples

- ✓ Factory Pattern Overview
- ✓ Factory Method vs Abstract Factory
- ✓ C++ Implementation Patterns
- ✓ Autonomous Vehicle Example
- ✓ Factory Pattern Benefits and Trade-offs

→ *Swipe to learn* →

Factory Pattern (Factory Method and Abstract Factory)

Theory

📌 1. Factory Pattern Overview (1/2)

Definition: Creational design pattern that provides an interface for creating objects without specifying exact classes, delegating instantiation to subclasses or factory methods.

Two Main Variants:

Pattern	Intent	Use Case
Factory Method	Define interface for creating object; subclasses decide which class to instantiate	Single product hierarchy, extension through inheritance
Abstract Factory	Create families of related objects without specifying concrete classes	Multiple related product hierarchies, consistent product families

Core Guarantee:

Requirement	Implementation	Purpose
Decoupling	Client code uses interfaces, not concrete classes	Reduce dependencies on specific implementations
Extensibility	Add new product types without modifying existing code	Open/Closed Principle
Consistency	Factory ensures related objects work together	Prevent incompatible product combinations
Centralized creation	All instantiation logic in one place	Easier testing, debugging, maintenance

When to Use Factory Pattern:

Common Use Cases:

```
class SensorFactory {           // ✅ Multiple sensor types (LIDAR, RADAR, Camera)
class VehicleFactory {           // ✅ Different vehicle configurations (Sedan, Truck, Bus)
class MessageParser {            // ✅ Various message formats (JSON, XML, Binary)
class UIComponentFactory {       // ✅ Platform-specific UI (Windows, Mac, Linux)
class DatabaseConnection {       // ✅ Multiple database backends (MySQL, PostgreSQL, MongoDB)
```

Factory Pattern (Factory Method and Abstract Factory)

Theory

📌 1. Factory Pattern Overview (2/2)

Inappropriate Uses:

```
class StringFactory {            // ❌ String is simple, no variants needed
class IntFactory {               // ❌ Primitive types don't need factories
class SingletonFactory {         // ❌ Singleton already manages its instantiation
```

Factory Pattern (Factory Method and Abstract Factory)

Theory

📌 2. Factory Method vs Abstract Factory (1/4)

Factory Method Pattern:

```
// Product interface
class Sensor {
public:
    virtual ~Sensor() = default;
    virtual double readValue() = 0;
    virtual std::string getType() const = 0;
};

// Concrete products
class LidarSensor : public Sensor {
public:
    double readValue() override { return 15.7; }
    std::string getType() const override { return "LIDAR"; }
};

class RadarSensor : public Sensor {
public:
    double readValue() override { return 8.3; }
    std::string getType() const override { return "RADAR"; }
};

// Creator with factory method
class SensorSystem {
public:
    virtual ~SensorSystem() = default;

    // Factory method - subclasses override to create specific sensors
    virtual std::unique_ptr<Sensor> createSensor() = 0;

    void processData() {
        auto sensor = createSensor();  // Use factory method
        double value = sensor->readValue();
        std::cout << sensor->getType() << ": " << value << "\n";
    }
};

// Concrete creators
class LidarSystem : public SensorSystem {
public:
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Theory

📌 2. Factory Method vs Abstract Factory (2/4)

[Code continues across slides]

```
// ...continued from previous slide
    std::unique_ptr<Sensor> createSensor() override {
        return std::make_unique<LidarSensor>();
    }
};

class RadarSystem : public SensorSystem {
public:
    std::unique_ptr<Sensor> createSensor() override {
        return std::make_unique<RadarSensor>();
    }
};
```

Factory Pattern (Factory Method and Abstract Factory)

Theory

📌 2. Factory Method vs Abstract Factory (3/4)

Abstract Factory Pattern:

```
// Product families: Sensor + Display + Logger

class Sensor {
public:
    virtual ~Sensor() = default;
    virtual double readValue() = 0;
};

class Display {
public:
    virtual ~Display() = default;
    virtual void show(double value) = 0;
};

class Logger {
public:
    virtual ~Logger() = default;
    virtual void log(const std::string& msg) = 0;
};

// Abstract factory interface
class SystemFactory {
public:
    virtual ~SystemFactory() = default;
    virtual std::unique_ptr<Sensor> createSensor() = 0;
    virtual std::unique_ptr<Display> createDisplay() = 0;
    virtual std::unique_ptr<Logger> createLogger() = 0;
};

// Concrete factory for Production environment
class ProductionFactory : public SystemFactory {
public:
    std::unique_ptr<Sensor> createSensor() override {
        return std::make_unique<HardwareSensor>();  // Real hardware
    }
    std::unique_ptr<Display> createDisplay() override {
        return std::make_unique<LCDDisplay>();      // Physical display
    }
    std::unique_ptr<Logger> createLogger() override {
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Theory

📌 2. Factory Method vs Abstract Factory (4/4)

[Code continues across slides]

```
// ...continued from previous slide
        return std::make_unique<FileLogger>();      // Log to file
    }
};

// Concrete factory for Testing environment
class TestFactory : public SystemFactory {
public:
    std::unique_ptr<Sensor> createSensor() override {
        return std::make_unique<MockSensor>();      // Simulated sensor
    }
    std::unique_ptr<Display> createDisplay() override {
        return std::make_unique<ConsoleDisplay>();  // Debug output
    }
    std::unique_ptr<Logger> createLogger() override {
        return std::make_unique<MemoryLogger>();    // In-memory logging
    }
};
```

Key Differences:

Aspect	Factory Method	Abstract Factory
Focus	Single product	Family of related products
Inheritance	Subclass decides what to create	Interface for creating multiple related objects
Complexity	Simpler, one factory method	More complex, multiple factory methods
Use when	Need to vary one product type	Need consistent families of products
Example	Document factory (PDF vs Word)	UI toolkit (Windows vs Mac buttons + windows + menus)

Factory Pattern (Factory Method and Abstract Factory)

Theory

📌 3. C++ Implementation Patterns (1/3)

A. Simple Function Factory (No Classes):

```
enum class SensorType { LIDAR, RADAR, CAMERA };

std::unique_ptr<Sensor> createSensor(SensorType type) {
    switch (type) {
        case SensorType::LIDAR:
            return std::make_unique<LidarSensor>();
        case SensorType::RADAR:
            return std::make_unique<RadarSensor>();
        case SensorType::CAMERA:
            return std::make_unique<CameraSensor>();
        default:
            throw std::invalid_argument("Unknown sensor type");
    }
}
```

B. Static Factory Method:

```
class Sensor {
public:
    enum class Type { LIDAR, RADAR, CAMERA };

    // Static factory method
    static std::unique_ptr<Sensor> create(Type type) {
        switch (type) {
            case Type::LIDAR:
                return std::make_unique<LidarSensor>();
            case Type::RADAR:
                return std::make_unique<RadarSensor>();
            case Type::CAMERA:
                return std::make_unique<CameraSensor>();
        }
    }

    virtual ~Sensor() = default;
    virtual double readValue() = 0;
};
```

Factory Pattern (Factory Method and Abstract Factory)

Theory

📌 3. C++ Implementation Patterns (2/3)

C. Registry-Based Factory (Runtime Registration):

```
class SensorFactory {
    using Creator = std::function<std::unique_ptr<Sensor>()>;
    std::map<std::string, Creator> registry;

public:
    void registerSensor(const std::string& name, Creator creator) {
        registry[name] = creator;
    }

    std::unique_ptr<Sensor> create(const std::string& name) {
        auto it = registry.find(name);
        if (it == registry.end()) {
            throw std::runtime_error("Unknown sensor: " + name);
        }
        return it->second();  // Call creator function
    }
};

// Registration
SensorFactory factory;
factory.registerSensor("lidar", []() { return std::make_unique<LidarSensor>(); });
factory.registerSensor("radar", []() { return std::make_unique<RadarSensor>(); });

// Usage
auto sensor = factory.create("lidar");
```

Factory Pattern (Factory Method and Abstract Factory)

Theory

📌 3. C++ Implementation Patterns (3/3)

D. Template-Based Factory:

```
template<typename ProductType>
class Factory {
public:
    template<typename ConcreteType, typename... Args>
    static std::unique_ptr<ProductType> create(Args&&... args) {
        return std::make_unique<ConcreteType>(std::forward<Args>(args)...);
    }
};

// Usage
auto lidar = Factory<Sensor>::create<LidarSensor>(/* params */);
auto radar = Factory<Sensor>::create<RadarSensor>(/* params */);
```

Factory Pattern (Factory Method and Abstract Factory)

Theory

📌 4. Autonomous Vehicle Example

Real-World Factory Use Case:

```
// Product hierarchy
class PerceptionAlgorithm {
public:
    virtual ~PerceptionAlgorithm() = default;
    virtual void processFrame(const Image& img) = 0;
    virtual std::vector<Detection> getDetections() const = 0;
};

class YOLODetector : public PerceptionAlgorithm { /* YOLO v8 */ };
class RCNNDetector : public PerceptionAlgorithm { /* Faster R-CNN */ };
class TransformerDetector : public PerceptionAlgorithm { /* Vision Transformer */ };

// Factory for different vehicle configurations
class PerceptionFactory {
public:
    enum class VehicleType { URBAN_TAXI, HIGHWAY_TRUCK, ROBOTAXI };

    static std::unique_ptr<PerceptionAlgorithm> createDetector(VehicleType type) {
        switch (type) {
            case VehicleType::URBAN_TAXI:
                // Fast inference for city driving (pedestrians, cyclists)
                return std::make_unique<YOLODetector>(/* optimized for speed */);

            case VehicleType::HIGHWAY_TRUCK:
                // Long-range detection for highways (vehicles, lane markers)
                return std::make_unique<RCNNDetector>(/* optimized for range */);

            case VehicleType::ROBOTAXI:
                // Highest accuracy for full autonomy
                return std::make_unique<TransformerDetector>(/* state-of-the-art */);
        }
    }
};

// Usage in vehicle initialization
auto detector = PerceptionFactory::createDetector(
    PerceptionFactory::VehicleType::ROBOTAXI
);
```

Factory Pattern (Factory Method and Abstract Factory)

Theory

📌 5. Factory Pattern Benefits and Trade-offs

Benefits:

Benefit	Explanation	Example
Open/Closed Principle	Add new types without modifying existing code	Add new sensor type by creating new class + factory entry
Single Responsibility	Creation logic separated from business logic	Sensor processing doesn't know about construction details
Dependency Inversion	Depend on abstractions, not concretions	Client code uses `Sensor*`, not `LidarSensor*`
Testability	Easy to inject mock implementations	Test factory returns `MockSensor` instead of `HardwareSensor`
Consistency	Factory ensures compatible object families	Production factory returns all hardware objects; test factory returns all mocks

Trade-offs:

Drawback	Explanation	Mitigation
Complexity	More classes and indirection	Use simple function factory for basic cases
Compile-time overhead	Virtual calls, pointer indirection	Consider templates for compile-time polymorphism
Memory overhead	Heap allocation via smart pointers	Use object pools for frequent allocations
Registration boilerplate	Manual registration in registry factories	Use macros or code generation

Factory Pattern (Factory Method and Abstract Factory)

Theory

📌 6. Why Factory Pattern Matters

Critical Concepts Demonstrated:

Concept	How Factory Tests It	Interview Relevance
Polymorphism	Client uses base pointer, factory returns derived	Core OOP understanding
Smart pointers	Ownership transfer via unique_ptr/shared_ptr	Modern C++ resource management
RAII	Factory returns RAII-wrapped resources	Exception safety, cleanup
Design patterns	Classic GoF pattern implementation	Software architecture knowledge
Decoupling	Client doesn't know concrete types	Maintainability, extensibility

Common Interview Questions:

- "When would you use Factory Method vs Abstract Factory?" (Single product vs product family)
- "How do you add new product types without breaking existing code?" (Open/Closed Principle)
- "What's the difference between Factory and Builder pattern?" (Object creation vs object construction)
- "How do you implement type-safe factories in C++?" (Templates, enums, std::variant)

Factory Pattern (Factory Method and Abstract Factory)

Edge Cases

📌 1. Exception Safety in Factory Methods (1/2)

```
// ❌ DANGEROUS: Exception during construction leaks resources
class SensorFactory {
public:
    static Sensor* createSensor(Type type) {  // ❌ Returns raw pointer
        Sensor* sensor = nullptr;
        switch (type) {
            case Type::LIDAR:
                sensor = new LidarSensor();  // May throw
                break;
            case Type::RADAR:
                sensor = new RadarSensor();
                break;
        }

        // If this throws, sensor leaks!
        sensor->initialize();  // ❌ May throw exception
        return sensor;
    }
};

// ✅ CORRECT: Use smart pointers for exception safety
class SensorFactory {
public:
    static std::unique_ptr<Sensor> createSensor(Type type) {
        auto sensor = [type]() -> std::unique_ptr<Sensor> {
            switch (type) {
                case Type::LIDAR:
                    return std::make_unique<LidarSensor>();
                case Type::RADAR:
                    return std::make_unique<RadarSensor>();
                default:
                    throw std::invalid_argument("Unknown type");
            }
        }();

        // Exception here doesn't leak - unique_ptr handles cleanup
        sensor->initialize();
        return sensor;  // ✅ Move semantics, no copy
    }
};
```

Factory Pattern (Factory Method and Abstract Factory)

Edge Cases

📌 1. Exception Safety in Factory Methods (2/2)

Why This Matters: Raw pointers in factories are dangerous. If initialization throws after allocation, memory leaks occur. Always return smart pointers from factories to ensure RAII cleanup.

Factory Pattern (Factory Method and Abstract Factory)

Edge Cases

📌 2. Object Slicing in Factory Returns

```
// ❌ BROKEN: Returning by value causes slicing
class SensorFactory {
public:
    static Sensor createSensor(Type type) {  // ❌ Returns by value!
        if (type == Type::LIDAR) {
            return LidarSensor();  // ❌ Sliced to Sensor base class
        }
        return RadarSensor();
    }
};

void useFactory() {
    Sensor sensor = SensorFactory::createSensor(Type::LIDAR);
    sensor.readValue();  // ❌ Calls Sensor::readValue(), not LidarSensor::readValue()
    // Derived class data lost due to slicing!
}

// ✅ CORRECT: Return pointer or reference
class SensorFactory {
public:
    static std::unique_ptr<Sensor> createSensor(Type type) {  // ✅ Pointer
        if (type == Type::LIDAR) {
            return std::make_unique<LidarSensor>();  // ✅ No slicing
        }
        return std::make_unique<RadarSensor>();
    }
};
```

Why This Matters: Returning polymorphic objects by value destroys the polymorphism through slicing. The derived class data is lost, and virtual functions don't work correctly. Always return pointers (preferably smart pointers) or references for polymorphic objects.

Factory Pattern (Factory Method and Abstract Factory)

Edge Cases

📌 3. Static Initialization Order with Registry Factories

```
// ❌ DANGEROUS: Registration happens during static initialization
class SensorFactory {
    static std::map<std::string, Creator>& getRegistry() {
        static std::map<std::string, Creator> registry;  // ✅ Function-local static
        return registry;
    }

public:
    static void registerSensor(const std::string& name, Creator creator) {
        getRegistry()[name] = creator;
    }
};

// ❌ PROBLEM: Order of static initialization across files is undefined
namespace {
    struct LidarRegistrar {
        LidarRegistrar() {
            // What if SensorFactory's static map not initialized yet?
            SensorFactory::registerSensor("lidar", []() {
                return std::make_unique<LidarSensor>();
            });
        }
    } lidarReg;  // Global static object
}
```

Solution: Use function-local static (Meyers Singleton) for the registry to avoid static initialization order fiasco. The registry is guaranteed to be initialized on first access.

Factory Pattern (Factory Method and Abstract Factory)

Edge Cases

📌 4. Type Erasure and Factory Return Types

```
// Challenge: Factory needs to return different types
class ComponentFactory {
public:
    // ❌ Can't have multiple return types in C++
    // auto create(Type type);  // What does this return?
};

// ✅ SOLUTION 1: Common base class (traditional polymorphism)
class Component { virtual ~Component() = default; };
class Sensor : public Component {};
class Actuator : public Component {};

std::unique_ptr<Component> create(Type type);

// ✅ SOLUTION 2: std::variant (type-safe union)
using ComponentVariant = std::variant<Sensor, Actuator, Display>;

ComponentVariant create(Type type) {
    switch (type) {
        case Type::SENSOR: return Sensor{};
        case Type::ACTUATOR: return Actuator{};
        case Type::DISPLAY: return Display{};
    }
}

// Usage with std::visit
std::visit([](auto&& component) {
    component.process();  // Works for all types with process()
}, factory.create(type));

// ✅ SOLUTION 3: std::any (runtime type erasure)
std::any create(Type type);  // Can return any type

auto sensor = std::any_cast<Sensor>(factory.create(Type::SENSOR));
```

Why This Matters: C++ is statically typed, so factories returning different unrelated types require careful design. Choose based on needs: inheritance for polymorphism, std::variant for type-safe known types, std::any for complete type erasure.

Factory Pattern (Factory Method and Abstract Factory)

Edge Cases

📌 5. Circular Dependencies in Factory Registration (1/2)

```
// ❌ PROBLEM: Factory A needs Factory B, Factory B needs Factory A
class SensorFactory {
public:
    static std::unique_ptr<Sensor> create(Type type) {
        // Needs ProcessorFactory to create associated processor
        auto processor = ProcessorFactory::create(type);  // ❌ Circular dependency
        return std::make_unique<Sensor>(std::move(processor));
    }
};

class ProcessorFactory {
public:
    static std::unique_ptr<Processor> create(Type type) {
        // Needs SensorFactory to create associated sensor
        auto sensor = SensorFactory::create(type);  // ❌ Circular dependency
        return std::make_unique<Processor>(std::move(sensor));
    }
};

// ✅ SOLUTION 1: Abstract Factory (create related objects together)
class SystemFactory {
public:
    struct Components {
        std::unique_ptr<Sensor> sensor;
        std::unique_ptr<Processor> processor;
    };

    static Components createSystem(Type type) {
        Components comp;
        comp.sensor = std::make_unique<Sensor>(type);
        comp.processor = std::make_unique<Processor>(type);
        // Wire them together
        comp.sensor->setProcessor(comp.processor.get());
        comp.processor->setSensor(comp.sensor.get());
        return comp;
    }
};

// ✅ SOLUTION 2: Two-phase initialization
auto sensor = SensorFactory::create(type);
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Edge Cases

📌 5. Circular Dependencies in Factory Registration (2/2)

[Code continues across slides]

```
// ...continued from previous slide
auto processor = ProcessorFactory::create(type);
sensor->setProcessor(processor.get());  // Wire after creation
processor->setSensor(sensor.get());
```

Why This Matters: Circular dependencies between factories indicate design problems. Use Abstract Factory to create related objects together, or use two-phase initialization (create-then-wire) to break the cycle.

Factory Pattern (Factory Method and Abstract Factory)

Edge Cases

📌 6. Thread Safety in Lazy Factory Initialization

```
// ❌ NOT THREAD-SAFE: Lazy initialization without synchronization
class SensorFactory {
    static std::map<std::string, std::unique_ptr<Sensor>> cache;

public:
    static Sensor* getSensor(const std::string& type) {
        // ❌ Race condition: multiple threads may create multiple instances
        if (cache.find(type) == cache.end()) {
            cache[type] = createSensor(type);  // Not atomic!
        }
        return cache[type].get();
    }
};

// ✅ CORRECT: Use mutex or std::call_once
class SensorFactory {
    static std::map<std::string, std::unique_ptr<Sensor>> cache;
    static std::mutex mtx;

public:
    static Sensor* getSensor(const std::string& type) {
        std::lock_guard<std::mutex> lock(mtx);

        auto it = cache.find(type);
        if (it == cache.end()) {
            cache[type] = createSensor(type);
            return cache[type].get();
        }
        return it->second.get();
    }
};
```

Why This Matters: Factories with caching or lazy initialization need thread safety. Without synchronization, multiple threads can create duplicate objects or corrupt internal state. Use mutex, std::call_once, or lock-free data structures for thread-safe factories.

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 1. Easy - Simple Function Factory (1/2)

This simple factory uses a free function with an enum to select the sensor type. The factory returns `unique_ptr<Sensor>` for automatic memory management. Clients don't need to know about concrete sensor classes.

```
#include <iostream>
#include <memory>
#include <string>

// Product interface
class Sensor {
public:
    virtual ~Sensor() = default;
    virtual double readValue() = 0;
    virtual std::string getType() const = 0;
};

// Concrete products
class LidarSensor : public Sensor {
public:
    double readValue() override {
        return 15.7;  // Simulated distance in meters
    }
    std::string getType() const override {
        return "LIDAR";
    }
};

class RadarSensor : public Sensor {
public:
    double readValue() override {
        return 8.3;  // Simulated distance in meters
    }
    std::string getType() const override {
        return "RADAR";
    }
};

class CameraSensor : public Sensor {
public:
    double readValue() override {
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 1. Easy - Simple Function Factory (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        return 120.5;  // Simulated brightness value
    }
    std::string getType() const override {
        return "CAMERA";
    }
};

// Simple function factory
enum class SensorType { LIDAR, RADAR, CAMERA };

std::unique_ptr<Sensor> createSensor(SensorType type) {
    switch (type) {
        case SensorType::LIDAR:
            return std::make_unique<LidarSensor>();
        case SensorType::RADAR:
            return std::make_unique<RadarSensor>();
        case SensorType::CAMERA:
            return std::make_unique<CameraSensor>();
        default:
            throw std::invalid_argument("Unknown sensor type");
    }
}

int main() {
    std::cout << "Simple Function Factory Example\n\n";

    // Create different sensors using factory
    auto lidar = createSensor(SensorType::LIDAR);
    auto radar = createSensor(SensorType::RADAR);
    auto camera = createSensor(SensorType::CAMERA);

    // Use sensors polymorphically
    std::cout << lidar->getType() << " reading: " << lidar->readValue() << "\n";
    std::cout << radar->getType() << " reading: " << radar->readValue() << "\n";
    std::cout << camera->getType() << " reading: " << camera->readValue() << "\n";

    return 0;
}
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 2. Mid - Factory Method Pattern (1/3)

Factory Method pattern delegates object creation to subclasses. The base class `SensorSystem` defines the factory method interface, and concrete systems (`LidarSystem` , `RadarSystem`) implement it to create specific sensors.

```
#include <iostream>
#include <memory>
#include <vector>

// Product interface
class Sensor {
public:
    virtual ~Sensor() = default;
    virtual void calibrate() = 0;
    virtual double readValue() = 0;
    virtual std::string getName() const = 0;
};

// Concrete products
class LidarSensor : public Sensor {
    double baseline = 0.0;

public:
    void calibrate() override {
        baseline = 0.5;  // Calibration offset
        std::cout << "  LIDAR calibrated (baseline: " << baseline << ")\n";
    }

    double readValue() override {
        return 15.7 - baseline;
    }

    std::string getName() const override {
        return "LIDAR-HD";
    }
};

class RadarSensor : public Sensor {
    double baseline = 0.0;

public:
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 2. Mid - Factory Method Pattern (2/3)

```
// ...continued from previous slide
    void calibrate() override {
        baseline = 0.3;
        std::cout << "  RADAR calibrated (baseline: " << baseline << ")\n";
    }

    double readValue() override {
        return 8.3 - baseline;
    }

    std::string getName() const override {
        return "RADAR-77GHz";
    }
};

// Creator class with factory method
class SensorSystem {
public:
    virtual ~SensorSystem() = default;

    // Factory method - subclasses override to create specific sensors
    virtual std::unique_ptr<Sensor> createSensor() = 0;

    // Template method using factory method
    void initializeAndRun() {
        std::cout << "\nInitializing sensor system...\n";

        auto sensor = createSensor();  // Factory method
        std::cout << "Created sensor: " << sensor->getName() << "\n";

        sensor->calibrate();

        std::cout << "Reading value: " << sensor->readValue() << "\n";
    }
};

// Concrete creators
class LidarSystem : public SensorSystem {
public:
    std::unique_ptr<Sensor> createSensor() override {
        return std::make_unique<LidarSensor>();
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 2. Mid - Factory Method Pattern (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    }
};

class RadarSystem : public SensorSystem {
public:
    std::unique_ptr<Sensor> createSensor() override {
        return std::make_unique<RadarSensor>();
    }
};

int main() {
    std::cout << "Factory Method Pattern Example\n";

    // Create different sensor systems
    std::vector<std::unique_ptr<SensorSystem>> systems;
    systems.push_back(std::make_unique<LidarSystem>());
    systems.push_back(std::make_unique<RadarSystem>());

    // Run each system - factory method creates appropriate sensor
    for (auto& system : systems) {
        system->initializeAndRun();
    }

    return 0;
}
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 3. Mid - Static Factory Method with Parameters (1/2)

Static factory methods provide named constructors with clear intent. Instead of overloaded constructors, factory methods have descriptive names that clarify what's being created.

```
#include <iostream>
#include <memory>
#include <string>

// Product with multiple construction options
class Sensor {
protected:
    std::string name;
    int sampleRate;

public:
    Sensor(const std::string& n, int rate) : name(n), sampleRate(rate) {}
    virtual ~Sensor() = default;

    virtual void displayInfo() const {
        std::cout << "  Sensor: " << name << ", Rate: " << sampleRate << "Hz\n";
    }

    // Static factory methods for different configurations
    static std::unique_ptr<Sensor> createHighPerformance() {
        return std::make_unique<Sensor>("HP-Sensor", 1000);
    }

    static std::unique_ptr<Sensor> createStandard() {
        return std::make_unique<Sensor>("STD-Sensor", 100);
    }

    static std::unique_ptr<Sensor> createLowPower() {
        return std::make_unique<Sensor>("LP-Sensor", 10);
    }

    static std::unique_ptr<Sensor> createCustom(const std::string& name, int rate) {
        if (rate <= 0 || rate > 10000) {
            throw std::invalid_argument("Invalid sample rate");
        }
        return std::make_unique<Sensor>(name, rate);
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 3. Mid - Static Factory Method with Parameters (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }
};

int main() {
    std::cout << "Static Factory Method with Parameters\n\n";

    // Create sensors using descriptive factory methods
    auto hp = Sensor::createHighPerformance();
    auto std = Sensor::createStandard();
    auto lp = Sensor::createLowPower();
    auto custom = Sensor::createCustom("CustomSensor", 500);

    std::cout << "Created sensors:\n";
    hp->displayInfo();
    std->displayInfo();
    lp->displayInfo();
    custom->displayInfo();

    return 0;
}
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 4. Advanced - Abstract Factory Pattern (1/4)

Abstract Factory creates families of related objects. The `ProductionFactory` creates hardware components for deployment, while `TestFactory` creates mock components for testing. The client code (`Application`) works with both seamlessly.

```
#include <iostream>
#include <memory>
#include <string>

// Product families: Sensor, Display, Logger

// Abstract products
class Sensor {
public:
    virtual ~Sensor() = default;
    virtual double readValue() = 0;
    virtual std::string getType() const = 0;
};

class Display {
public:
    virtual ~Display() = default;
    virtual void show(const std::string& msg) = 0;
};

class Logger {
public:
    virtual ~Logger() = default;
    virtual void log(const std::string& msg) = 0;
};

// Concrete products for Production family
class HardwareSensor : public Sensor {
public:
    double readValue() override { return 42.5; }  // Real hardware read
    std::string getType() const override { return "HardwareSensor"; }
};

class LCDDisplay : public Display {
public:
    void show(const std::string& msg) override {
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 4. Advanced - Abstract Factory Pattern (2/4)

```
// ...continued from previous slide
        std::cout << "[LCD] " << msg << "\n";
    }
};

class FileLogger : public Logger {
public:
    void log(const std::string& msg) override {
        std::cout << "[FileLog] " << msg << "\n";  // Simulated file write
    }
};

// Concrete products for Testing family
class MockSensor : public Sensor {
    double mockValue = 10.0;

public:
    double readValue() override { return mockValue; }  // Predictable test value
    std::string getType() const override { return "MockSensor"; }
};

class ConsoleDisplay : public Display {
public:
    void show(const std::string& msg) override {
        std::cout << "[Console] " << msg << "\n";
    }
};

class MemoryLogger : public Logger {
    std::vector<std::string> logs;

public:
    void log(const std::string& msg) override {
        logs.push_back(msg);
        std::cout << "[MemLog] " << msg << " (stored in memory)\n";
    }
};

// Abstract factory interface
class SystemFactory {
public:
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 4. Advanced - Abstract Factory Pattern (3/4)

```
// ...continued from previous slide
    virtual ~SystemFactory() = default;
    virtual std::unique_ptr<Sensor> createSensor() = 0;
    virtual std::unique_ptr<Display> createDisplay() = 0;
    virtual std::unique_ptr<Logger> createLogger() = 0;
};

// Concrete factory for Production
class ProductionFactory : public SystemFactory {
public:
    std::unique_ptr<Sensor> createSensor() override {
        return std::make_unique<HardwareSensor>();
    }

    std::unique_ptr<Display> createDisplay() override {
        return std::make_unique<LCDDisplay>();
    }

    std::unique_ptr<Logger> createLogger() override {
        return std::make_unique<FileLogger>();
    }
};

// Concrete factory for Testing
class TestFactory : public SystemFactory {
public:
    std::unique_ptr<Sensor> createSensor() override {
        return std::make_unique<MockSensor>();
    }

    std::unique_ptr<Display> createDisplay() override {
        return std::make_unique<ConsoleDisplay>();
    }

    std::unique_ptr<Logger> createLogger() override {
        return std::make_unique<MemoryLogger>();
    }
};

// Client code - uses abstract factory
class Application {
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 4. Advanced - Abstract Factory Pattern (4/4)

[Code continues across slides]

```
// ...continued from previous slide
    std::unique_ptr<Sensor> sensor;
    std::unique_ptr<Display> display;
    std::unique_ptr<Logger> logger;

public:
    Application(SystemFactory& factory) {
        sensor = factory.createSensor();
        display = factory.createDisplay();
        logger = factory.createLogger();
    }

    void run() {
        logger->log("Application started");

        double value = sensor->readValue();
        std::string msg = "Sensor reading: " + std::to_string(value);

        display->show(msg);
        logger->log(msg);
    }
};

int main() {
    std::cout << "Abstract Factory Pattern Example\n\n";

    std::cout << "=== PRODUCTION MODE ===\n";
    ProductionFactory prodFactory;
    Application prodApp(prodFactory);
    prodApp.run();

    std::cout << "\n=== TEST MODE ===\n";
    TestFactory testFactory;
    Application testApp(testFactory);
    testApp.run();

    return 0;
}
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 5. Advanced - Registry-Based Factory with Lambda Creators (1/3)

Registry-based factory allows runtime registration of creators. Useful when loading sensor types from configuration files or plugins. The factory uses `std::function` to store any callable that creates sensors.

```
#include <iostream>
#include <memory>
#include <map>
#include <functional>
#include <string>

// Product interface
class Sensor {
public:
    virtual ~Sensor() = default;
    virtual std::string getType() const = 0;
    virtual double readValue() = 0;
};

// Concrete products
class LidarSensor : public Sensor {
public:
    std::string getType() const override { return "LIDAR"; }
    double readValue() override { return 15.7; }
};

class RadarSensor : public Sensor {
public:
    std::string getType() const override { return "RADAR"; }
    double readValue() override { return 8.3; }
};

class CameraSensor : public Sensor {
public:
    std::string getType() const override { return "CAMERA"; }
    double readValue() override { return 120.5; }
};

// Registry-based factory
class SensorFactory {
    using Creator = std::function<std::unique_ptr<Sensor>()>;
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 5. Advanced - Registry-Based Factory with Lambda Creators (2/3)

```
// ...continued from previous slide

    // Use function-local static to avoid initialization order fiasco
    static std::map<std::string, Creator>& getRegistry() {
        static std::map<std::string, Creator> registry;
        return registry;
    }

public:
    // Register a creator function
    static void registerSensor(const std::string& type, Creator creator) {
        getRegistry()[type] = creator;
    }

    // Create sensor by type name
    static std::unique_ptr<Sensor> create(const std::string& type) {
        auto& registry = getRegistry();
        auto it = registry.find(type);

        if (it == registry.end()) {
            throw std::runtime_error("Unknown sensor type: " + type);
        }

        return it->second();  // Call creator function
    }

    // List available sensor types
    static std::vector<std::string> getAvailableTypes() {
        std::vector<std::string> types;
        for (const auto& pair : getRegistry()) {
            types.push_back(pair.first);
        }
        return types;
    }
};

// Self-registering helper
template<typename T>
struct SensorRegistrar {
    SensorRegistrar(const std::string& type) {
        SensorFactory::registerSensor(type, []() {
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 5. Advanced - Registry-Based Factory with Lambda Creators (3/3)

[Code continues across slides]

```
// ...continued from previous slide
            return std::make_unique<T>();
        });
    }
};

int main() {
    std::cout << "Registry-Based Factory Example\n\n";

    // Register sensors using lambdas
    SensorFactory::registerSensor("lidar", []() {
        return std::make_unique<LidarSensor>();
    });

    SensorFactory::registerSensor("radar", []() {
        return std::make_unique<RadarSensor>();
    });

    SensorFactory::registerSensor("camera", []() {
        return std::make_unique<CameraSensor>();
    });

    // List available types
    std::cout << "Available sensor types:\n";
    for (const auto& type : SensorFactory::getAvailableTypes()) {
        std::cout << "  - " << type << "\n";
    }
    std::cout << "\n";

    // Create sensors by string name (useful for config files)
    std::vector<std::string> sensorTypes = {"lidar", "radar", "camera"};

    for (const auto& type : sensorTypes) {
        auto sensor = SensorFactory::create(type);
        std::cout << sensor->getType() << " reading: "
                  << sensor->readValue() << "\n";
    }

    return 0;
}
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 7. Advanced - Template-Based Factory with Perfect Forwarding (1/3)

Template factory with perfect forwarding allows creating objects with any constructor signature. The factory forwards all arguments to the concrete type's constructor without copying.

```
#include <iostream>
#include <memory>
#include <string>
#include <utility>

// Product base
class Sensor {
protected:
    std::string id;

public:
    Sensor(const std::string& identifier) : id(identifier) {
        std::cout << "  Creating sensor: " << id << "\n";
    }

    virtual ~Sensor() = default;
    virtual std::string getType() const = 0;
};

// Concrete products with different constructors
class LidarSensor : public Sensor {
    int beamCount;

public:
    LidarSensor(const std::string& id, int beams)
        : Sensor(id), beamCount(beams) {
        std::cout << "    LIDAR with " << beamCount << " beams\n";
    }

    std::string getType() const override { return "LIDAR"; }
};

class RadarSensor : public Sensor {
    double frequency;
    bool longRange;

// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 7. Advanced - Template-Based Factory with Perfect Forwarding (2/3)

```
// ...continued from previous slide
public:
    RadarSensor(const std::string& id, double freq, bool lr)
        : Sensor(id), frequency(freq), longRange(lr) {
        std::cout << "    RADAR at " << frequency << "GHz, "
                  << (longRange ? "long" : "short") << " range\n";
    }

    std::string getType() const override { return "RADAR"; }
};

class CameraSensor : public Sensor {
    std::string resolution;
    int fps;

public:
    CameraSensor(const std::string& id, const std::string& res, int framerate)
        : Sensor(id), resolution(res), fps(framerate) {
        std::cout << "    Camera " << resolution << " @ " << fps << "fps\n";
    }

    std::string getType() const override { return "CAMERA"; }
};

// Template-based factory with perfect forwarding
template<typename BaseType>
class Factory {
public:
    template<typename ConcreteType, typename... Args>
    static std::unique_ptr<BaseType> create(Args&&... args) {
        return std::make_unique<ConcreteType>(std::forward<Args>(args)...);
    }
};

int main() {
    std::cout << "Template Factory with Perfect Forwarding\n\n";

    // Create different sensors with varying constructor parameters
    auto lidar = Factory<Sensor>::create<LidarSensor>("LIDAR-001", 64);

    auto radar = Factory<Sensor>::create<RadarSensor>("RADAR-002", 77.0, true);
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 7. Advanced - Template-Based Factory with Perfect Forwarding (3/3)

[Code continues across slides]

```
// ...continued from previous slide

    auto camera = Factory<Sensor>::create<CameraSensor>("CAM-003", "4K", 60);

    std::cout << "\nCreated sensors:\n";
    std::cout << "  " << lidar->getType() << "\n";
    std::cout << "  " << radar->getType() << "\n";
    std::cout << "  " << camera->getType() << "\n";

    return 0;
}
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 8. Performance - Object Pool Factory (Reusable Objects) (1/4)

Object pool factory reuses expensive objects instead of creating new ones each time. Critical for real-time systems (autonomous vehicles) where allocation latency is unacceptable. Pool reduces allocations from O(n) to O(1) after warmup.

```
#include <iostream>
#include <memory>
#include <vector>
#include <queue>
#include <mutex>

// Expensive object to create
class SensorData {
    std::vector<double> pointCloud;  // Large data structure

public:
    SensorData() {
        pointCloud.reserve(100000);  // Reserve large buffer
        std::cout << "  [Expensive] Allocating SensorData (100K points)\n";
    }

    void reset() {
        pointCloud.clear();
    }

    void addPoint(double value) {
        pointCloud.push_back(value);
    }

    size_t size() const { return pointCloud.size(); }
};

// Object pool factory - recycles expensive objects
class SensorDataPool {
    std::queue<std::unique_ptr<SensorData>> pool;
    std::mutex mtx;
    size_t created = 0;
    size_t recycled = 0;

public:
    // Acquire object from pool (or create new if pool empty)
// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 8. Performance - Object Pool Factory (Reusable Objects) (2/4)

```
// ...continued from previous slide
    std::unique_ptr<SensorData> acquire() {
        std::lock_guard<std::mutex> lock(mtx);

        if (pool.empty()) {
            created++;
            return std::make_unique<SensorData>();
        }

        recycled++;
        auto obj = std::move(pool.front());
        pool.pop();
        obj->reset();
        return obj;
    }

    // Release object back to pool
    void release(std::unique_ptr<SensorData> obj) {
        std::lock_guard<std::mutex> lock(mtx);
        pool.push(std::move(obj));
    }

    void printStats() const {
        std::cout << "\nPool Statistics:\n";
        std::cout << "  Objects created: " << created << "\n";
        std::cout << "  Objects recycled: " << recycled << "\n";
        std::cout << "  Objects in pool: " << pool.size() << "\n";
    }
};

// RAII wrapper for auto-release
class PooledSensorData {
    std::unique_ptr<SensorData> data;
    SensorDataPool* pool;

public:
    PooledSensorData(std::unique_ptr<SensorData> d, SensorDataPool* p)
        : data(std::move(d)), pool(p) {}

    ~PooledSensorData() {
        if (data && pool) {
// ...continues on next slide
```

📌 8. Performance - Object Pool Factory (Reusable Objects) (3/4)

```
// ...continued from previous slide
            pool->release(std::move(data));
        }
    }

    SensorData* operator->() { return data.get(); }
    SensorData& operator*() { return *data; }
};

int main() {
    std::cout << "Object Pool Factory Pattern\n\n";

    SensorDataPool pool;

    std::cout << "First batch (creates new objects):\n";
    {
        auto data1 = pool.acquire();
        auto data2 = pool.acquire();
        auto data3 = pool.acquire();

        data1->addPoint(1.0);
        data2->addPoint(2.0);
        data3->addPoint(3.0);

        std::cout << "  Using 3 objects...\n";

        // Return to pool
        pool.release(std::move(data1));
        pool.release(std::move(data2));
        pool.release(std::move(data3));
    }

    std::cout << "\nSecond batch (recycles from pool):\n";
    {
        auto data1 = pool.acquire();  // Recycled
        auto data2 = pool.acquire();  // Recycled
        auto data3 = pool.acquire();  // Recycled
        auto data4 = pool.acquire();  // New (pool exhausted)

        std::cout << "  Using 4 objects (3 recycled, 1 new)...\n";

// ...continues on next slide
```

Factory Pattern (Factory Method and Abstract Factory)

Code Examples

📌 8. Performance - Object Pool Factory (Reusable Objects) (4/4)

[Code continues across slides]

```
// ...continued from previous slide
        pool.release(std::move(data1));
        pool.release(std::move(data2));
        pool.release(std::move(data3));
        pool.release(std::move(data4));
    }

    pool.printStats();

    return 0;
}
```

What You Learned

Key Takeaways

- ✓ Factory Pattern Overview
- ✓ Factory Method vs Abstract Factory
- ✓ C++ Implementation Patterns
- ✓ Autonomous Vehicle Example

→ Swipe to learn →